Assignment : 10.2

Name : D. Akash

HT.NO : 2303A51422

Batch : 21

Task Description -1(Error Detection and Correction)

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Expected Output-1:

Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

Code :

```
'''Task Description -1(Error Detection and Correction)
Task:
Use AI to analyze a Python script and correct all syntax and logical
errors.
Sample Input Code:
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
Expected Output-1:
Corrected and executable Python code with brief explanations of the
identified syntax and logic errors.
Corrected Code:
```python'''
def calculate_total(nums):
    sum = 0
    for n in nums:
        sum += n
    return sum
# Explanation of Errors:
# 1. Missing colon (:) after the function definition and the for loop.
# 2. The variable 'total' was used in the return statement,
# but it should be 'sum' to return the correct total of the numbers.

# example usage:
numbers = [1, 2, 3, 4, 5]
total = calculate_total(numbers)
print("Total:", total)  # Output: Total: 15

```

Output :

```
[Running] python -u "c:\Users\akash\OneDrive\Desktop\training\DSA.PY"
Total: 15
```

Observation : In this task, AI was used to analyze the given Python script and identify syntax and logical errors. The AI corrected issues such as missing colons, incorrect variable usage, and improper loop syntax. It also replaced the incorrect return variable with the correct one. The final output produced a clean and executable Python program with explanations for each correction.
```

Task Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

def findSum(a,b):return a+b

print(findSum(5,10))

Expected Output-2:

Well-structured, consistently formatted Python code following standard

style conventions.

Code :

```
[Running] python -u "c:\Users\akash\OneDrive\Desktop\training\DSA.PY"
Total: 15
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\akash\OneDrive\Desktop\training> & C:
15
PS C:\Users\akash\OneDrive\Desktop\training>
```

Observation :

In this task, AI refactored the Python code to follow standard coding conventions such as proper indentation, spacing, and function formatting. The code was rewritten to improve readability and maintain consistency with common Python style guidelines. The

function definition and print statement were separated and formatted clearly. The result was a well-structured and easy-to-understand Python program.

Task Description -3(Code Clarity Improvement)

Task:

Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):
return x-y*2
print(f(10,3))
```

Expected Output-3:

Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.

Code :

```python
'''Task Description -3(Code Clarity Improvement)
Task:
Use AI to improve code readability without changing its functionality.
Sample Input Code:
def f(x,y):
return x-y*2
print(f(10,3))
Expected Output-3:
Python code rewritten with meaningful function and variable names,
proper indentation, and improved clarity.'''

def calculate_difference(x, y):
    """
    This function takes two numbers as input and returns the result of subtracting
    twice the second number from the first number.

    Parameters:
    x (int): The first number.
    y (int): The second number.

    Returns:
    int: The result of x minus twice y.
    """
    return x - (y * 2)
# Example usage
result = calculate_difference(10, 3)
print(result)
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PO

PS C:\Users\akash\OneDrive\Desktop\training> &
4
PS C:\Users\akash\OneDrive\Desktop\training>
```

Observation :

In this task, AI improved the readability of the code without changing its functionality. Meaningful function and variable names were introduced to replace unclear ones. Proper indentation and formatting were applied to make the logic easier to understand. The final code became more descriptive and maintainable while producing the same output.

Task Description -4(Structural Refactoring)

Task:

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

print("Hello Ram")

print("Hello Sita")

print("Hello Ravi")

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.

Code :

```python
'''Task Description -4(Structural Refactoring)
Task:
Use AI to refactor repetitive code into reusable functions.
Sample Input Code:
print("Hello Ram")
print("Hello Sita")
print("Hello Ravi")
Expected Output-4:
Modular Python code using reusable functions to eliminate repetition.'''
def greet(name):
    print(f"Hello {name}")
names = ["Ram", "Sita", "Ravi"]
for name in names:
    greet(name)
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR QUERY RESULTS

```
[Running] python -u "c:\Users\akash\OneDrive\Desktop\training\DSA.PY"
Hello Ram
Hello Sita
Hello Ravi
```

Observation :

In this task, AI transformed repetitive print statements into a reusable function. This approach reduced code duplication and improved modularity. The function was designed to accept different names as parameters and display greetings. As a result, the code became shorter, cleaner, and easier to reuse or modify.

Task Description -5(Efficiency Enhancement)

Task:

Use AI to optimize Python code for better performance.

Sample Input Code:

numbers = [ ]

for i in range(1, 500000):

numbers.append(i * i)

print(len(numbers))

Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

Code :

```
'''Task Description -5(Efficiency Enhancement)
Task:
Use AI to optimize Python code for better performance.
Sample Input Code:
numbers = [ ]
for i in range(1, 500000):
numbers.append(i * i)
print(len(numbers))
Expected Output-5:
Optimized Python code that achieves the same result with improved
performance.'''
# Optimized Python code using list comprehension for better performance
numbers = [i * i for i in range(1, 500000)]
print(len(numbers))
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PO

PS C:\Users\akash\OneDrive\Desktop\training> &
499999
PS C:\Users\akash\OneDrive\Desktop\training>
```

Observation :

In this task, AI optimized the Python code to improve performance when generating large numbers. Instead of using a loop with repeated append operations, a more efficient approach such as list comprehension was used. This reduced execution time and improved code efficiency. The optimized code still produced the same result while using fewer